CODEWITHSHIVAH

4 NOVEMBER 2026

CLASS 03 NOTES

NumPy — arrays, masks, broadcasting and axes

01	Why a Python list is not enough
02	Creating arrays and reading their shape
03	Indexing, slicing and boolean masks
04	Broadcasting and the axis rule
05	Three exercises, with answers
06	NumPy on the student dataset

Machine Learning: Build → Understand → Deploy

31 live classes // 2 November – 29 December 2026

learn.rajeevshivah.me

CLASS 03 · 01

FIRST, WHY A PYTHON LIST IS NOT ENOUGH

PYTHON LIST

A flexible box that can hold anything — numbers, text, other lists — scattered around memory. Flexible, but slow, and it cannot do maths on itself.

```
nums = list(range(1_000_000))

total = 0
for n in nums:
    total += n          # one million Python steps
```

This works. It is also slow, uses far more memory than it needs, and cannot do maths on a whole column at once. Let us time it against the alternative.

CLASS 03 · 02

THE SAME WORK, IN ONE CALL

NUMPY

A Python library for working with whole blocks of numbers at once instead of one at a time. Every ML library you will touch — Pandas, scikit-learn, PyTorch — is built on top of it.

`np.arange(n)` — makes an array of 0, 1, 2 ... up to n, the array version of `range()`.

```
import numpy as np
import time

nums = list(range(1_000_000))
arr  = np.arange(1_000_000)

t0 = time.time(); sum(nums);  print("list :", time.time()-t0)
t0 = time.time(); arr.sum();  print("numpy:", time.time()-t0)
```

```
list : 0.0512
numpy: 0.0006        # roughly 80x faster
```

NumPy stores numbers in one unbroken block of memory and does the loop in C, not in Python. That is the whole trick.

CREATING ARRAYS

ARRAY

A grid of numbers, all of the same type, stored in one unbroken block of memory. That is why NumPy is fast — and why you cannot mix text and numbers in one array.

`array()` from a list · `zeros()` / `ones()` filled · `arange()` counts by a **step** · `linspace()` gives an exact **count**

```
import numpy as np

a = np.array([2, 4, 6, 8])              # from a list
b = np.zeros(5)                         # [0. 0. 0. 0. 0.]
c = np.ones((2, 3))                     # 2 rows, 3 columns of 1.0
d = np.arange(0, 10, 2)                 # [0 2 4 6 8]
e = np.linspace(0, 1, 5)                # 5 evenly spaced, 0 to 1

print(a)
print(e)
```

```
[2 4 6 8]
[0.   0.25 0.5  0.75 1.  ]
```

`arange` takes a step. `linspace` takes a count. Mixing them up is the most common first-week mistake.

CLASS 03 · 04

THREE THINGS TO CHECK ON EVERY ARRAY

THE FOUR PROPERTIES

`.shape` — how many rows and columns, as a pair: `(2, 3)` means 2 rows, 3 columns.

`.ndim` — how many dimensions: 1 is a single row of numbers, 2 is a table.

`.dtype` — what kind of number is inside: `int64` whole numbers, `float64` decimals.

`.size` — how many numbers in total.

```
m = np.array([[1, 2, 3],
              [4, 5, 6]])

print(m.shape)   # (2, 3)  -> 2 rows, 3 columns
print(m.ndim)    # 2       -> it is a matrix, not a vector
print(m.dtype)   # int64   -> whole numbers
print(m.size)    # 6       -> total elements
```

shape is the one you will read a thousand times this term. Ninety percent of the errors you hit in scikit-learn are shape errors. When something breaks, print the shape first, always.

CLASS 03 · 05

YOUR TURN · 3 MINUTES

BUILD IT AND INSPECT IT

1. Make a 1D array of 5, 10, 15 ... up to 50. Do not type the numbers.
2. Reshape it into **2 rows**.
3. Print its `shape`, `ndim` and `dtype`.

Type it yourself. Do not copy from the recording later — the muscle memory is the point.

CLASS 03 · 06

CHECK YOUR ANSWER

```
a = np.arange(5, 55, 5)
print(a)                          # [ 5 10 15 20 25 30 35 40 45 50]

m = a.reshape(2, 5)
print(m.shape, m.ndim, m.dtype)   # (2, 5) 2 int64
```

Why `55` and not `50`? `arange` stops *before* the end value. Getting 45 instead of 50 is the most common slip here.

CLASS 03 · 07

INDEXING AND SLICING

INDEXING & SLICING

Indexing = one item by position. **Slicing** = a range, `start:stop`, stop never included. In 2D: `[row, column]`; a bare `:` means "all".

```
a = np.array([10, 20, 30, 40, 50])

a[0]        # 10          first
a[-1]       # 50          last
a[1:4]      # [20 30 40]  from 1 up to (not including) 4
a[:3]       # [10 20 30]
a[::2]      # [10 30 50]  every second element

m = np.array([[1, 2, 3],
              [4, 5, 6]])

m[0, 2]     # 3           row 0, column 2
m[:, 1]     # [2 5]       every row, column 1  -> one feature
m[1, :]     # [4 5 6]     row 1, all columns   -> one sample
```

`m[:, 1]` is how you pull out a single feature, and `m[1, :]` is how you pull out a single sample. You will write both constantly.

CLASS 03 · 08

BOOLEAN MASKS — THE MOST USEFUL IDEA IN NUMPY

BOOLEAN MASK

Comparing an array returns a **whole array of True/False**, one per element — that is the mask. Put it in brackets and NumPy keeps only the True positions.

```
scores = np.array([45, 82, 67, 91, 38, 74])

mask = scores >= 60
print(mask)           # [False  True  True  True False  True]

print(scores[mask])   # [82 67 91 74]   only the passing scores
print(scores[scores >= 60].mean())   # 78.5

# combine conditions: use & and |, and bracket each one
mid = scores[(scores >= 60) & (scores < 85)]
print(mid)            # [82 67 74]
```

Use `&` and `|`, never `and` / `or`, and bracket every condition. Forgetting the brackets is the error you will all hit tonight.

CLASS 03 · 09

YOUR TURN · 4 MINUTES

ASK THE ARRAY THREE QUESTIONS

```
marks = np.array([45, 82, 67, 91, 38, 74, 59, 88])
```

1. How many students scored **60 or above**?
2. What is the average of **only those** scores?
3. Print the scores that are **60 or above but below 85**.

No loops. No `if`. One line each.

CLASS 03 · 10

CHECK YOUR ANSWER

```
marks = np.array([45, 82, 67, 91, 38, 74, 59, 88])

print((marks >= 60).sum())                  # 5
print(marks[marks >= 60].mean())            # 80.4
print(marks[(marks >= 60) & (marks < 85)])  # [82 67 74]
```

Look at the first line again. `marks >= 60` gives True/False, and `.sum()` counts the Trues because True is 1. That trick — summing a condition to count it — you will use every single week from now on.

CLASS 03 · 11

MATHS ON A WHOLE ARRAY AT ONCE

VECTORISATION
One operation applied to every element at once, instead of looping. You say what to do; NumPy runs the loop in C. `.std()` below is standard deviation — how spread out the numbers are.

```
prices = np.array([120, 340, 250, 90])

prices * 2          # [240 680 500 180]
prices + 10         # [130 350 260 100]
prices / 100        # [1.2 3.4 2.5 0.9]

gst = prices * 0.18
total = prices + gst
print(total)        # [141.6 401.2 295.  106.2]

print(prices.mean(), prices.max(), prices.std().round(2))
```

```
[141.6 401.2 295.  106.2]
200.0 340 95.39
```

No loop anywhere. This is called **vectorisation**, and from today it is how you write numeric Python.

CLASS 03 · 12

BROADCASTING

BROADCASTING

When two arrays are different sizes, NumPy quietly stretches the smaller one so the shapes line up — then does the maths. Nothing is actually copied in memory, so it stays fast.

It is the reason `array + 10` works at all: the single 10 is stretched to every cell.

```
m = np.array([[1, 2, 3],
              [4, 5, 6]])          # shape (2, 3)

m + 10          # the 10 is stretched to every cell
# [[11 12 13]
#  [14 15 16]]

col_means = m.mean(axis=0)         # [2.5 3.5 4.5]  shape (3,)
centered = m - col_means           # (2,3) - (3,) works
print(centered)
```

```
[[-1.5 -1.5 -1.5]
 [ 1.5  1.5  1.5]]
```

That last operation — subtracting the column mean from every row — is exactly what feature scaling does. You will meet it again in class 10.

CLASS 03 · 13

THE RULE, IN ONE LINE

Compare shapes from the right. Two dimensions are compatible if they are **equal**, or if one of them is **1**.

SHAPE A	SHAPE B	RESULT
(2, 3)	(3,)	(2, 3) — works
(2, 3)	(2, 1)	(2, 3) — works
(2, 3)	(2,)	error — 3 and 2 do not match

When you get a broadcasting error, print both shapes and line them up from the right. The mismatch will be obvious.

CLASS 03 · 14

AXIS=0 AND AXIS=1 — GET THIS RIGHT ONCE

AXIS

A direction through the array. `axis=0` runs **down** the rows, `axis=1` runs **across** the columns.

Whichever axis you name is the one that gets squashed flat. Name `axis=0` and the rows vanish, leaving one number per column.

```
m = np.array([[1, 2, 3],
              [4, 5, 6]])

m.sum()            # 21        everything
m.sum(axis=0)      # [5 7 9]   down the rows -> one value per COLUMN
m.sum(axis=1)      # [6 15]    across columns -> one value per ROW
```

The trick: **axis is the dimension that disappears.** `axis=0` collapses the rows, so you are left with one number per column. In a dataset, a column is a feature — so `axis=0` is what you almost always want.

CLASS 03 · 15

YOUR TURN · 4 MINUTES

THE ONE THAT MATTERS

```
m = np.arange(12).reshape(3, 4)
```

1. The mean of **each column**.
2. The sum of **each row**.
3. Subtract the column means from every row — **in one line**.

If you get question 3, you have understood both broadcasting and axes. That is the whole class.

CLASS 03 · 16

CHECK YOUR ANSWER

```
m = np.arange(12).reshape(3, 4)

print(m.mean(axis=0))      # [4. 5. 6. 7.]   one value per column
print(m.sum(axis=1))       # [ 6 22 38]      one value per row

print(m - m.mean(axis=0))
# [[-4. -4. -4. -4.]
#  [ 0.  0.  0.  0.]
#  [ 4.  4.  4.  4.]]
```

That last line is **mean centering**. Shape `(3,4)` minus shape `(4,)`, and NumPy lines them up from the right. Half of feature scaling is this single line.

CLASS 03 · 17

TWO MORE YOU WILL USE CONSTANTLY

RESHAPE & WHERE

`.reshape(rows, cols)` — same numbers, rearranged into a new shape. The total count must still match. `-1` means "you work this one out for me".

`np.where(condition, a, b)` — the array version of if/else. Where the condition is True you get `a`, where it is False you get `b`.

```
a = np.arange(12)
a.reshape(3, 4)        # 3 rows, 4 columns
a.reshape(-1, 1)       # 12 rows, 1 column  (-1 means "work it out")

# scikit-learn demands a 2D X. This is the fix you will Google
# in week three, so learn it now:
X = a.reshape(-1, 1)
print(X.shape)         # (12, 1)

marks = np.array([45, 82, 67, 38])
np.where(marks >= 60, "pass", "fail")
# array(['fail', 'pass', 'pass', 'fail'], dtype='<U4')
```

CLASS 03 · 18

RANDOM NUMBERS, AND WHY YOU SET A SEED

SEED

Computers cannot make truly random numbers — they follow a recipe from a starting value. That starting value is the seed.

Fix the seed and you get the same "random" numbers every single run. That is what makes your results **reproducible** — anyone running your notebook sees exactly what you saw.

```
rng = np.random.default_rng(seed=42)

rng.integers(0, 100, size=5)      # same five numbers every run
rng.normal(50, 10, size=5)        # mean 50, std 10
```

Set a seed in every notebook you share. Without it, your results change on each run and nobody — including you next week — can reproduce them. In an interview, "did you set a random seed?" is a real question.

CLASS 03 · 19

NOW POINT IT AT REAL DATA

Same `students.csv` the model beat my rule on last night. No Pandas yet — pure NumPy.

```
import numpy as np

data = np.genfromtxt("students.csv", delimiter=",", skip_header=1)
print(data.shape)                 # (200, 4)

hours      = data[:, 0]           # column 0 = one feature
attendance = data[:, 1]
prev_marks = data[:, 2]
passed     = data[:, 3]           # the answer column

print(hours[passed == 1].mean().round(2))   # 6.68
print(hours[passed == 0].mean().round(2))   # 3.42
```

```
(200, 4)
6.68
3.42
```

CLASS 03 · 20

TWO NUMBERS, ONE INSIGHT

	PASSED	FAILED
Hours studied	6.68	3.42
Attendance	76.7	69.0

Students who passed studied roughly **twice** as long. Attendance barely moved — about 8 points apart.

> Remember what the model said last night? `hours_studied +0.813`, `attendance +0.066`. The model worked that out on its own. You just confirmed it by hand, with two lines of NumPy. **That is what this tool is for.**

CLASS 03 · 21

THREE HABITS TO LEAVE WITH

1. **No loops over data.** If you are writing `for` over an array, there is almost always an array operation that does it.
2. **Print the shape when something breaks.** Not the values. The shape.
3. **Think in columns, not cells.** A feature is a column. An operation applies to the whole column at once.

CLASS 03 · 22

BEFORE THURSDAY

1. Create an array of 20 random exam marks between 0 and 100 with `np.random.randint`, then print mean, max, min and standard deviation.
2. Use a boolean mask to print only the marks above the mean, and count them.
3. Build `np.arange(20).reshape(4, 5)`. Print its second column, its last row, and its column means.
4. Subtract the column means from every row in one line, and explain in a sentence why that works.

> Post your notebook in the group. If something errors, post the error too — reading other people's errors is half of learning this.

Thursday: Pandas, on a file I have broken on purpose.
